

Curso técnico em Desenvolvimento de Sistemas – 3º ano Ensino Médio
Disciplina Desenvolvimento de Sistemas – Professor Mizael Carlos

Atividade – Padrões de Projeto

1. Explique o que é um Design Pattern (Padrão de Projeto). Por que se diz que um padrão de projeto "não é código pronto", mas sim uma solução reutilizável para um problema recorrente?
2. Cite e explique três benefícios de utilizar padrões de projeto no desenvolvimento de um sistema. Para cada benefício, dê um exemplo de situação em que ele faria diferença em uma equipe de desenvolvimento.
3. Os padrões de projeto são organizados em três famílias: Criacionais, Estruturais e Comportamentais. Explique qual tipo de problema cada família resolve e cite um padrão de exemplo para cada uma delas.
4. Explique o objetivo do padrão Singleton e cite duas situações de um sistema real em que ele seria adequado.
5. Por que o padrão Singleton precisa impedir que outras partes do sistema criem novas instâncias da classe livremente? Que problemas poderiam surgir se a aplicação abrisse várias conexões com o banco de dados ao mesmo tempo, sem controle?
6. Frameworks como o Laravel já oferecem, por meio do Service Container, uma forma de registrar uma classe como Singleton. Explique qual vantagem existe em usar esse recurso do framework em vez de implementar o padrão manualmente em cada classe.
7. Explique o padrão Factory Method. Por que é vantajoso que o código que utiliza um objeto não precise conhecer a classe concreta que será criada? Use como exemplo um sistema que envia notificações por e-mail, SMS ou push.
8. Explique o padrão Builder. Por que um objeto com muitos parâmetros opcionais, criado por um único construtor extenso, torna o código confuso e difícil de manter? Explique também o conceito de interface fluente (method chaining) e como ela melhora a leitura do código.

9. Explique o padrão Adapter usando uma analogia do dia a dia (por exemplo, um adaptador de tomada). Depois, explique como esse padrão permite trocar o fornecedor de um gateway de pagamento sem alterar o restante do sistema.

10. O padrão Decorator adiciona responsabilidades a um objeto sem alterar sua classe original. Explique por que usar herança para combinar funcionalidades opcionais (por exemplo, café com leite, com chocolate, com leite e chocolate) pode gerar uma "explosão de subclasses" e como o Decorator evita esse problema.

11. Um dos diferenciais do Decorator é permitir adicionar comportamento em tempo de execução, e não em tempo de compilação. Explique a diferença entre esses dois momentos e por que essa flexibilidade é útil em um sistema de pedidos de uma cafeteria.

12. Explique o padrão Facade. No Laravel, recursos como Storage, Mail e Cache são exemplos de Facades. O que acontece "por trás" de uma Facade e qual benefício ela traz para o desenvolvedor que escreve as regras do sistema?

13. Os padrões Adapter, Decorator e Facade pertencem à família dos Estruturais e todos "envolvem" outras classes. Explique a diferença de objetivo entre eles: o que cada um faz com a classe que envolve?

14. Explique o padrão Strategy. Por que uma longa sequência de condições (if/else) que escolhe o comportamento com base em um tipo é considerada um sinal de que o Strategy poderia ser aplicado?

15. Um sistema de e-commerce aplica regras de desconto diferentes conforme a época do ano (Natal, Black Friday, sem desconto). Descreva, com suas palavras, como o padrão Strategy organizaria essas regras e o que aconteceria no sistema quando uma nova promoção precisasse ser criada.

16. O padrão Observer define uma dependência "um-para-muitos". Explique o que isso significa e descreva um cenário de sistema (por exemplo, quando um pedido é pago) em que várias partes precisam reagir a uma mesma mudança sem se conhecerem.

17. Explique o padrão Template Method. Em um sistema que gera vários tipos de relatório (vendas, estoque, clientes), o que ficaria definido na classe base e o que cada tipo de relatório precisaria definir por conta própria? Justifique.

18. Explique o Repository Pattern e por que ele isola a lógica de acesso a dados dos Controllers. Se os dados de um sistema passassem a vir de uma API externa em vez do banco de dados, qual seria a vantagem de já utilizar esse padrão?

19. Padrões de projeto nem sempre devem ser usados. Explique o que é over-engineering e descreva duas situações em que aplicar um padrão de projeto seria um erro. Comente a frase: "Resolva o problema primeiro, nomeie o padrão depois."

20. Leia o cenário abaixo e, para cada necessidade, indique qual padrão de projeto você aplicaria e justifique sua escolha.

- a) Manter uma única conexão com o banco de dados compartilhada por toda a aplicação.
- b) Calcular o frete de formas diferentes conforme a transportadora escolhida pelo cliente.
- c) Enviar um e-mail, registrar um log e atualizar o estoque sempre que um pedido for pago.
- d) Integrar uma biblioteca de nota fiscal cuja interface é diferente da usada no sistema.